

Verifica TPSIT - Thread	NOME		VOTO
	DATA	CLASSE	

LEGENDA: ☐ Risposta singola: una sola opzione ☐ Risposta multipla Ordinamento

Nell'esecuzione **concorrente** più attività sono attive nello stesso intervallo di tempo, alternandosi prima che l'una o l'altra abbiano concluso la propria esecuzione. Attenzione: *concorrente* è molto diverso da *contemporaneamente*, infatti su un processore single-core è possibile avere più attività concorrenti ma è impossibile avere più attività in esecuzione contemporaneamente, si tratta sempre di un'alternanza di chi sta usando la CPU. Attenzione: un processo/thread in **esecuzione** significa che sta usando la CPU. Un processo/thread può essere attivo ma non utilizzare la CPU nel momento attuale.

Se due thread sono concorrenti su un singolo processore, si eseguono esattamente nello stesso istante. 1

Se ho _____ thread ho _____ linee di esecuzione contemporaneamente. 2

Spiega con parole tue la differenza tra operazione CPU bound e IO bound. Fai un esempio per ciascuna.

3

Su un singolo processore, due thread si alternano rapidamente. Come si definisce correttamente questa esecuzione?

☐ Sequenziale, perché un solo processore è coinvolto

☐ Parallela, perché il processore gestisce più thread

☐ Concorrente, perché entrambi i thread sono attivi nello stesso intervallo di tempo

☐ Contemporanea, perché i thread girano nello stesso istante

4

I thread introducono un problema difficile da _____ in quanto è necessario immaginarsi più thread in esecuzione contemporaneamente. Questo implica che ci sono più _____ contemporaneamente, ed è quindi impossibile _____ quale sarà la _____. Il problema introdotto dall'uso dei thread si chiama _____.

5

Più thread dello stesso processo non possono mai essere concorrenti. 6

```

1 import threading
2 def chiedi_numero():
3     input("Inserisci un numero:")
4 t = threading.Thread(target=chiedi_numero)
5 t.start()
    
```

Quanti thread ci sono durante l'esecuzione del precedente script?

☐ 0

☐ 1

☐ Nessuna delle precedenti.

☐ 2

7

Per ottenere il nome del thread attualmente attivo si usa la funzione `threading.` _____(). 8

```
1 task1()
2 task2()
```

Il precedente codice:

☐ task1 e task2 utilizzano sicuramente la CPU per il 100% del tempo della loro esecuzione.

☐ È un esempio di programmazione sequenziale.

☐ I thread non possono permettere di portare avanti insieme i due task contemporaneamente.

☐ È impossibile eseguire task2 fintanto che task1 non è terminato.

9

In un PC single-core, due task IO-Bound impiegano 10 secondi ciascuno di esecuzione. Eseguendo ciascun task in un thread dedicato, dopo quanto tempo entrambi terminano la loro esecuzione?

☐ 10 secondi

☐ tra 10 e 20 secondi

☐ 40 secondi

☐ 20 secondi

10

Quale delle seguenti è una funzione tipicamente IO bound?

☐ Invio dati su internet

☐ Calcolo di una divisione.

☐ Lettura dati su Hard Disk.

☐ Compressione/decompressione video.

11

In un programma sequenziale i task vengono eseguiti uno _____ l'altro, mentre con il multi-threading possono essere eseguiti in modo _____.

12

```
1 import threading, time
2
3 def stampa_tempo_trascorso():
4     tempo_iniziale = time.time()
5     while True:
6         print(time.time() - tempo_iniziale)
7         time.sleep(10)
8
9 def chiedi_numero():
10     while True:
11         x = input("Inserisci un numero:")
12
13 t1 = threading.Thread(target=stampa_tempo_trascorso)
14 t2 = threading.Thread(target=chiedi_numero)
15 t1.start()
16 t1.join()
17 t2.start()
18 t2.join()
```

Seleziona tutte le affermazioni corrette riguardo il precedente script.

☐ Il thread `t2` non viene mai avviato

☐ `stampa_tempo_trascorso` stampa ogni 10 secondi il tempo trascorso dall'avvio della funzione

☐ Entrambe le funzioni contengono un ciclo infinito

☐ I thread `t1` e `t2` vengono eseguiti in modo concorrente

☐ `time.sleep(10)` mette in pausa il thread principale per 10 secondi

☐ `t2` viene avviato prima di `t1`

13

Qual è l'errore critico presente nel precedente script e perché?

14

In un PC single-core, due task CPU-Bound impiegano 10 secondi ciascuno di esecuzione. Eseguendo ciascun task in un thread dedicato, dopo quanto tempo entrambi terminano la loro esecuzione?

☐ 40 secondi

☐ 20 secondi

☐ tra 10 e 20 secondi

☐ 10 secondi

15

Perché è più difficile fare il debug di programmi multi-thread?

☐ Perché non basta sapere la linea di esecuzione per sapere quale thread si sta eseguendo.

☐ Perché Python non supporta il debug di thread

☐ Perché la prossima linea di esecuzione potrebbe non essere quella successiva

☐ Perché ho più linee di esecuzione contemporaneamente

16

Perché usare il multi-threading per task IO bound può migliorare le prestazioni, mentre per task CPU bound il beneficio è limitato?

☐ Un task CPU bound occupa costantemente la CPU, limitando il beneficio dell'esecuzione concorrente di altri thread.

☐ Il multi-threading migliora sempre le prestazioni, indipendentemente dal tipo di task.

☐ Durante un task IO bound il thread è spesso in attesa di un evento esterno, per cui la CPU è disponibile per altri thread.

☐ Alternare più thread CPU bound in modo concorrente velocizza i calcoli complessivi.

17

Seleziona le affermazioni vere.

☐ I thread servono anche a facilitare la scrittura del codice per suddividerlo in "sotto-processi".

☐ In un processo sequenziale la sequenza della linea di esecuzione non è sempre prevedibile.

☐ La sequenza della linea di esecuzione con più thread è prevedibile.

☐ I thread sono inutili nelle CPU single-core.

☐ Un processo può contenere al massimo un thread.

☐ Un processo contiene minimo un thread.

18

In Python la libreria per creare thread si chiama _____.

19

Un'applicazione CPU bound è limitata dalla velocità della _____, mentre un'applicazione IO bound è limitata dalla velocità dei dispositivi di _____.

20